You

I have a javascript function that looks like this currently:

```
function doGet(e) {
  Logger.log(" doGet triggered with parameters: " + JSON.stringify(e.parameter));

  const name = decodeURIComponent(e.parameter.name || "Customer");
  const email = decodeURIComponent(e.parameter.email || "");
  const productsParam = e.parameter.products || "";
  const selectedProducts = productsParam.split(",").map(p => p.trim());

  Logger.log(" Customer name: " + name);
  Logger.log(" Email: " + email);
  Logger.log(" Selected products: " + JSON.stringify(selectedProducts));

  const productMap = getProductDataMap();
  Logger.log(" Loaded product data map with keys: " + Object.keys(productMap).join(", "));

  const sizeDataMap = {};
  let html = `<h2>Complete Your Order, ${name}</h2>`;
  html += `<form id="orderForm">`;

  function generateSizeBlock(productId, sizes, blockIndex) {
    return `
    <div class="size-block" id="${productId}_block_${blockIndex}" style="margin-bottom:10px;">
      <label>Size:</label>
      <select name="${productId}_size_${blockIndex}" required>
        <option value="">Select size</option>
        ${sizes.map(size => `<option>${size}</option>`).join("")}
      </select>
      <label>Qty:</label>
      <input type="number" name="${productId}_qty_${blockIndex}" min="1" required>
      <button type="button" onclick="removeSizeBlock('${productId}_block_${blockIndex}')">✖ Remove</button>
    </div> `;
  }

  selectedProducts.forEach((product, index) => {
    const productData = productMap[product] || {};
    const imageUrl = productData.image || "";
    const sizes = productData.sizes || ["YS", "YM", "YL", "S", "M", "L", "XL"];
    const productId = `product_${index}`;

    sizeDataMap[productId] = sizes;

    Logger.log(` Generating form block for product: ${product}`);
    Logger.log(` Image URL: ${imageUrl}`);
    Logger.log(` Sizes: ${JSON.stringify(sizes)}`);

    html += `
      <div style="margin-bottom: 30px; border-bottom: 1px solid #ccc; padding-bottom: 15px;">
        <h3>${product}</h3>
        ${imageUrl ? `<img src="${imageUrl}" style="width:200px; max-width:100%; margin-bottom:10px;"><br>` :
"<h4>No Image Available</h4>"}
        <div id="${productId}_sizeBlocks">
          ${generateSizeBlock(productId, sizes, 0)}
        </div>
        <button type="button" onclick="addSizeBlock('${productId}')">+ Add Another Size</button>
        <input type="hidden" name="${productId}_name" value="${product}">
      </div> `;
    });
```

html += ` <input type="hidden" name="name" value="${name}"> <input type="hidden" name="email" value="${email}"> <input type="hidden" name="productCount" value="${selectedProducts.length}"> <button type="submit">Submit Order</button> </form> <div id="responseMsg" style="margin-top:20px; font-weight:bold;"></div> <script> const sizeDataMap = ${JSON.stringify(sizeDataMap)}; const sizeBlockCounters = {}; window.onload = function () { for (const productId in sizeDataMap) { sizeBlockCounters[productId] = 1; } }; function addSizeBlock(productId) { const index = sizeBlockCounters[productId]++; const sizes = sizeDataMap[productId]; let optionsHtml = '<option value="">Select size</option>'; sizes.forEach(size => { optionsHtml += '<option>' + size + '</option>'; }); const block = document.createElement("div"); block.className = "size-block"; block.id = productId + '_block_' + index; block.style.marginBottom = "10px"; block.innerHTML = \` <label>Size:</label> <select name="\${productId}_size_\${index}" required> \${optionsHtml} </select> <label>Qty:</label> <input type="number" name="\${productId}_qty_\${index}" min="1" required> <button type="button" onclick="removeSizeBlock('\${productId}_block_\${index}')">❌ Remove</button> \`; document.getElementById(productId + "_sizeBlocks").appendChild(block); } function removeSizeBlock(blockId) { const el = document.getElementById(blockId); if (el) el.remove(); } </script> `; html += <script> document.getElementById('orderForm').addEventListener('submit', async function(event) { event.preventDefault(); const formData = new FormData(this); try { google.script.run .withSuccessHandler(function(result) { document.getElementById('responseMsg').innerText = result; document.getElementById('orderForm').style.display = 'none'; }) .withFailureHandler(function(error) { document.getElementById('responseMsg').innerText = '❌ Form submission failed.'; console.error(error); }) .handleFormSubmission(formData); const result = await response.text(); document.getElementById('responseMsg').innerText = result; this.style.display = 'none'; } catch (error) { console.error('Form submission failed:', error); document.getElementById('responseMsg').innerText = '❌ Failed to submit form.'; } }); </script> ; Logger.log("☑ HTML generation complete. Returning form."); return HtmlService.createHtmlOutput(html) .setTitle("Select Sizes & Quantities") .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL); }

I want to replace the line "const formData = new FormData(this);" with something like "const formData = $(e)" but I'm not sure for the syntax of how it can be used to treat *e* as a variable from my function. I don't want to modify any of the form handling logic or anything, I just want the syntax for integrating a variable into that line.

Copilot

Great question! If you're looking to use jQuery to reference the event target (the form element) instead of this, you can simply do this:

```
const formData = new FormData($(event.target)[0]);
```

Here's why this works:

- *event.target* refers to the element that triggered the event—in this case, the form.
- *$(event.target)* wraps the form in a jQuery object.
- [0] accesses the raw DOM element from the jQuery object so that *FormData()* can use it.

So that one-liner slots right in as a jQuery-flavored replacement without changing any of the form logic. Let me know if you want help tweaking anything else—like styling the response message or making the form submission feel snappier.

You

Almost there, I don't want to use *FormData* at all, I just want *$(e)* there but I want the js to treat it as a variable and not a string literal or anything like that

Copilot

Got it! If you're aiming to use $(e) where e is a variable—like the event object from your function—you just need to make sure e is defined and passed in correctly. For example:

```
document.getElementById('orderForm').addEventListener('submit', function(e) {
  const $form = $(e.target); // jQuery-wrapped form element
  // Now you can interact with $form using jQuery
});
```

In this case, *e.target* refers to the form element that triggered the submit event, and wrapping it with $() gives you jQuery power 🎸. Just don't forget that you'll need jQuery loaded in the page for this to work.

So if you're trying to use *$(e)* meaning "treat e as a variable, not a literal," you're already on the right path—just make sure you're accessing *e.target* (or the relevant DOM element) and not using quotes. Let me know if you want help chaining jQuery methods from it or extracting field values!

You

With this the variables with my *$* sign appear the same colour as the rest of the inserted js suggesting that the js will interpret them as string literals:

```
html += `
  <script>
    document.getElementById('orderForm').addEventListener('submit', async function(event) {
    event.preventDefault();
    const $formData = $(e);
    try {
      google.script.run
      .withSuccessHandler(function(result) {
        document.getElementById('responseMsg').innerText = result;
        document.getElementById('orderForm').style.display = 'none';
      })
      .withFailureHandler(function(error) {
        document.getElementById('responseMsg').innerText = '❌ Form submission failed.';
        console.error(error);
      })
      .handleFormSubmission($formData);
    const result = await response.text();
    document.getElementById('responseMsg').innerText = result;
    this.style.display = 'none';
    } catch (error) {
      console.error('Form submission failed:', error);
      document.getElementById('responseMsg').innerText = '❌ Failed to submit form.';
    }
  });
  </script> `
;
```

Copilot

You're super close—great eye spotting the syntax detail! The issue is that you're embedding JavaScript inside a template literal for HTML generation, and within that context, $(e) is being treated as plain text instead of referencing an actual variable. To ensure the variable event is used properly inside your embedded script, update the line like this:

```
const $formData = $(event.target);
```

This avoids using an undeclared e and ensures you're using the event object already passed into your handler function. *$(event.target)* correctly wraps the submitted form in a jQuery object.

Also, the coloring issue you mentioned (everything looking like a string literal) is probably just due to how your code editor highlights JavaScript inside a string—it doesn't affect how the browser interprets it as actual executable code.

If you're interested, I could also help modularize this embedded JavaScript for better readability and maintainability. Want me to show you how?

You

(Now Friday, Jul 25)
How can I modify my *doPost* method to output the text "Thanks for confirming your order. A confirmation email has been sent to your contact email. If there's any issues, feel free to contact us." on the html webpage as a heading 2?

Copilot

You can do this by returning an HTML response from your *doPost* function that includes the desired message wrapped in an *<h2>* tag. Here's a simple example of how to do that:

```
function doPost(e) {
  const htmlOutput = HtmlService.createHtmlOutput(
    '<h2>Thanks for confirming your order. A confirmation email has been sent to your contact email. If there\'s any issues, feel free to contact us.</h2>'
  );

  return htmlOutput.setTitle("Order Confirmation")
                   .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}
```

✨ A couple notes:

- The backslash *(\')* escapes the single quote inside the string.
- You can style the *<h2>* with inline CSS or wrap it in a fuller HTML layout if you'd like it to match your page's design.

Want help polishing the layout or embedding it in a branded confirmation page? I'd be happy to help you finesse the experience!

You

Sorry this is the method, modify this:

```
function doPost(e, emailAddress) {
  const params = e.parameter;
  Logger.log("🧾 Raw POST params: " + JSON.stringify(params));

  const customerName = params.name || "Customer";
  const customerEmail = emailAddress || "pythonprogramsha@gmail.com";
  const productCount = parseInt(params.productCount, 10);
  Logger.log(`👤 Customer: ${customerName} (${customerEmail}), Products: ${productCount}`);

  let summary = `🛍️ Order from ${customerName}\n\n`;

  for (let i = 0; i < productCount; i++) {
    const productName = params[`product_${i}_name`];
    Logger.log(`📦 Processing: ${productName}`);
    if (!productName) continue;

    summary += `  ◇ ${productName}\n`;
    let sizeQtyPairs = [];
    let j = 0;

    while (true) {
      const sizeKey = `product_${i}_size_${j}`;
      const qtyKey = `product_${i}_qty_${j}`;
      if (!(sizeKey in params) || !(qtyKey in params)) break;

      const size = params[sizeKey];
      const qty = params[qtyKey];
      Logger.log(`  • Pair ${j}: ${size} × ${qty}`);

      if (size && qty) {
        sizeQtyPairs.push(`  • Size: ${size}, Qty: ${qty}`);
      }
      j++;
    }

    if (sizeQtyPairs.length > 0) {
```

```
    summary += sizeQtyPairs.join("\n") + "\n\n";
  } else {
    summary += ` ⚠️ No sizes/quantities provided.\n\n`;
  }
 }

 Logger.log("📬 Summary:\n" + summary);

 // Owner email
 MailApp.sendEmail({
  to: "akhtarhasan2005@gmail.com",
  subject: `New Kit Order from ${customerName}`,
  body: summary
 });

 // Customer confirmation
 if (customerEmail && customerEmail.includes("@")) {
  MailApp.sendEmail({
   to: customerEmail,
   subject: `We received your order, ${customerName}!`,
   body: `Hi ${customerName},\n\nThanks for completing your kit order. Here's what we
received:\n\n${summary}\nBest,\nAPX Performance`
  });
 } else {
  Logger.log("⚠️ No valid customer email provided.");
 }

 return ContentService.createTextOutput("☑️ Order submitted successfully! Thank you.");
}
```

Copilot

To have the *doPost* method display your confirmation message as an *<h2>* heading on the resulting webpage, you'll want to replace the current *ContentService.createTextOutput(...)* with an HTML output. Here's your modified method with that change:

```
function doPost(e, emailAddress) {
 const params = e.parameter;
 Logger.log("🎱 Raw POST params: " + JSON.stringify(params));

 const customerName = params.name || "Customer";
 const customerEmail = emailAddress || "pythonprogramsha@gmail.com";
 const productCount = parseInt(params.productCount, 10);
 Logger.log(`🕵️ Customer: ${customerName} (${customerEmail}), Products: ${productCount}`);

 let summary = `🛍️ Order from ${customerName}\n\n`;

 for (let i = 0; i < productCount; i++) {
  const productName = params[`product_${i}_name`];
  Logger.log(`📦 Processing: ${productName}`);
  if (!productName) continue;

  summary += ` ◇ ${productName}\n`;
  let sizeQtyPairs = [];
  let j = 0;

  while (true) {
   const sizeKey = `product_${i}_size_${j}`;
   const qtyKey = `product_${i}_qty_${j}`;
   if (!(sizeKey in params) || !(qtyKey in params)) break;

   const size = params[sizeKey];
   const qty = params[qtyKey];
```

```
    Logger.log(`  • Pair ${j}: ${size} × ${qty}`);

    if (size && qty) {
      sizeQtyPairs.push(`  • Size: ${size}, Qty: ${qty}`);
    }
    j++;
   }

   if (sizeQtyPairs.length > 0) {
    summary += sizeQtyPairs.join("\n") + "\n\n";
   } else {
    summary += `  ⚠ No sizes/quantities provided.\n\n`;
   }
  }

  Logger.log("📋 Summary:\n" + summary);

  // Owner email
  MailApp.sendEmail({
   to: "akhtarhasan2005@gmail.com",
   subject: `New Kit Order from ${customerName}`,
   body: summary
  });

  // Customer confirmation
  if (customerEmail && customerEmail.includes("@")) {
   MailApp.sendEmail({
    to: customerEmail,
    subject: `We received your order, ${customerName}!`,
    body: `Hi ${customerName},\n\nThanks for completing your kit order. Here's what we
received:\n\n${summary}\nBest,\nAPX Performance`
   });
  } else {
   Logger.log("⚠ No valid customer email provided.");
  }

  const htmlContent = `
   <html>
    <head>
     <title>Order Confirmation</title>
     <style>
      body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; }
      h2 { color: #2E8B57; }
     </style>
    </head>
    <body>
     <h2>Thanks for confirming your order. A confirmation email has been sent to your contact email.<br>If there's
any issues, feel free to contact us.</h2>
    </body>
   </html>
  `;

  return HtmlService.createHtmlOutput(htmlContent)
          .setTitle("Order Confirmation")
          .setXFrameOptionsMode(HtmlService.XFrameOptionsMode.ALLOWALL);
}
```

This update gives your users a warm visual thank-you after placing an order—nice touch for boosting trust and user experience 👌

Would you like to embed the APX branding (like a logo or colors) into this confirmation page? I can help you mock that up too.

You

I still get this on the screen only after submitting the form:
Complete Your Order, Costello FC

When this is my full code:
[Full code was here, cba putting it here again]

Copilot

Ah, I see exactly what's happening—and you're close. The issue isn't with your *doPost* method (which is set up correctly to return the confirmation HTML), but with **how it's being handled inside the *doGet*-returned page**.

Right now, your *doGet* method builds the entire form and sets up client-side JavaScript to submit it via *google.script.run.handleFormSubmission(...)*. That part is great. But in your success handler:

```
.withSuccessHandler(function(result) {
  document.getElementById('responseMsg').innerText = result;
  document.getElementById('orderForm').style.display = 'none';
})
```

You're treating the result like a **text string**, even though your *doPost* returns **HTML content**.

☑ **Here's how to fix it:**
Change your success handler so that it updates the page with the HTML content returned from *doPost*:

```
.withSuccessHandler(function(result) {
  document.body.innerHTML = result.getContent();
})
```

This replaces the current page content with the confirmation message your *doPost* sends. 🎉

✨ **Bonus Suggestion:**

If you'd rather insert the confirmation message in a specific section (instead of replacing the full page), wrap your HTML output inside a div and inject it like:

```
document.getElementById('responseMsg').innerHTML = result.getContent();
```

Let me know which experience you're going for—and if you want the confirmation to feel more branded or dynamic, I can help dress it up!

You

Ok, now however I get this error:
Uncaught TypeError: Cannot read properties of null (reading 'getContent')
at :47:46
at eh (3165903335-mae_html_user_bin_i18n_mae_html_user__en_gb.js:119:417)
at 3165903335-mae_html_user_bin_i18n_mae_html_user__en_gb.js:35:132
at di.N (3165903335-mae_html_user_bin_i18n_mae_html_user__en_gb.js:143:374)
at Td (3165903335-mae_html_user_bin_i18n_mae_html_user__en_gb.js:69:477)
at a (3165903335-mae_html_user_bin_i18n_mae_html_user__en_gb.js:67:52)

With this code:

```
document.getElementById('orderForm').addEventListener('submit', async function(event) {
  event.preventDefault();
  const formData = new FormData(this);

  try {
    google.script.run
      .withSuccessHandler(function(result) {
        document.body.innerHTML = result.getContent();
    })
```

```
    .withFailureHandler(function(error) {
      document.getElementById('responseMsg').innerText = '❌ Form submission failed.';
      console.error(error);
    })
    .handleFormSubmission(Object.fromEntries(formData), ${email});
  } catch (error) {
    console.error('Form submission failed:', error);
    document.getElementById('responseMsg').innerText = '❌ Failed to submit form.';
  }
});
```

Copilot

The error you're seeing is happening because the *result* object returned by *handleFormSubmission* is not an instance of *HtmlOutput*, so trying to call *result.getContent()* throws that TypeError.

👉 In Google Apps Script, objects passed from server-side to client-side via *google.script.run* don't retain their class methods (like *.getContent()*). They're serialized as plain JavaScript objects or strings.

Since you're already returning a full HTML string in your *doPost*, you can skip *.getContent()* and use result directly:

```
.withSuccessHandler(function(result) {
  document.body.innerHTML = result; // Just use result, not result.getContent()
})
```

That should wipe out the error and cleanly replace the page content with your confirmation message. If you ever want more control—like inserting that message into a section of the page rather than swapping everything—I can help you refactor accordingly.

Want to add a little APX logo flair or some fade-in animation to make that confirmation more visually satisfying? I've got ideas.

You

Ok so now there's no errors, however the screen just goes blank after successfully submitting the form, the html text doesn't display

Copilot

Ah, I see the trouble—the form submits successfully, but then the page goes blank instead of showing your confirmation message. You're returning full HTML from *doPost*, but on the client side, you're injecting result into *document.body.innerHTML*, assuming it's just plain markup. The catch? When Apps Script sends back an *HtmlOutput*, the client-side *google.script.run* receives it as a **special object**, not a raw HTML string.

Here's how to fix it cleanly:

☑ **Update *doPost* to return a string instead of *HtmlOutput***

Change the last lines of *doPost* to:

```
return `
  <h2 style="color:#2E8B57; text-align:center; margin-top:50px;">
    Thanks for confirming your order. A confirmation email has been sent to your contact email.<br>
    If there's any issues, feel free to contact us.
  </h2>
`;
```

No need to return a full HTML page from doPost, because you're injecting this snippet directly into the page via JavaScript.

☑ **Confirm your client-side handler is ready for this string**

```
.withSuccessHandler(function(result) {
  document.body.innerHTML = result; // 'result' is now a plain HTML string
})
```

☑ Why this works

You're already within a full HTML shell rendered by *doGet*. Injecting another entire HTML document via *HtmlOutput* inside it isn’t the right fit. Instead, keeping it to just the HTML markup (string) lets you swap out the page content smoothly.

Want to make that confirmation message animate in, fade out the form gracefully, or include a team logo for ✨ style points? I’ve got ideas.

Conversation by Fifa Utuber with Copilot on Wednesday, Jul 23 2025